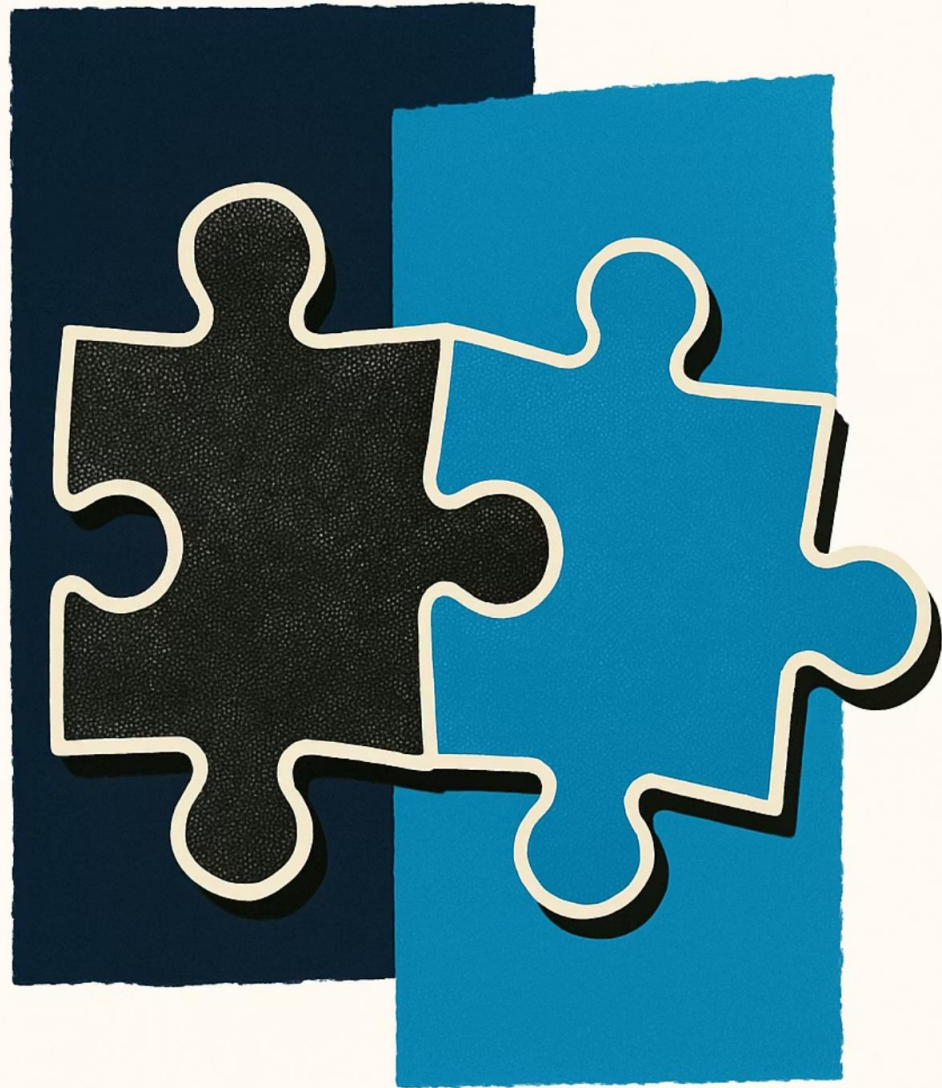


Angular Moderno

06 - Comunicação entre Componentes



Comunicação entre Componentes

Aprenda como componentes Angular trocam dados e eventos entre si, utilizando `@Input` e `@Output` com `EventEmitter` para construir aplicações mais dinâmicas e organizadas.

Angular Moderno

06 - Comunicação entre Componentes

Nesta aula, você vai entender como componentes Angular se comunicam, tornando suas aplicações mais interativas e bem estruturadas.



O que você vai aprender

Ao final desta aula, você será capaz de conectar componentes Angular de forma eficiente e organizada.

1

Enviar dados entre componentes

Transmitir informações de um componente para outro de forma direta.

2

Capturar eventos de filhos

Ouvir e reagir a ações disparadas por componentes filhos.

3

Usar @Input e @Output

Aplicar os decorators essenciais de comunicação do Angular.

4

Trabalhar com EventEmitter

Emitir e capturar eventos personalizados entre componentes.

5

Boas práticas

Organizar o código com responsabilidades bem separadas.

Antes de Começar

Para acompanhar esta aula sem dificuldades, verifique se o ambiente da aula anterior está pronto e funcionando corretamente.

Projeto da aula anterior

O projeto criado na aula 5 deve estar funcionando sem erros no seu editor.

Lista de produtos visível

A diretiva `@for` deve estar renderizando a lista de produtos na tela.

Servidor rodando

O comando `ng serve` deve estar ativo e o app acessível no navegador.

 Abra o terminal e execute `ng serve` antes de continuar.

O Problema Atual

Até este ponto do curso, você criou componentes que funcionam de forma **completamente independente**. Cada componente vive no seu próprio mundo, sem trocar informações com os outros.

Componentes que não se comunicam limitam o que você pode construir. Aplicações reais precisam que partes diferentes trabalhem juntas.

Imagine uma lista de produtos onde cada item precisa informar ao componente pai qual produto foi selecionado — isso não é possível sem comunicação entre componentes.



O que Vamos Resolver

Ao final desta aula, seus componentes vão deixar de ser ilhas isoladas e passar a se comunicar de forma estruturada e eficiente.

Componente envia dados

Um componente poderá passar informações diretamente para outro componente dentro da mesma árvore de componentes.

Componente responde a ações

Um componente filho poderá notificar o pai quando o usuário realizar uma ação, como clicar em um botão ou selecionar um item.

Tipos de Comunicação em Angular

No Angular, a comunicação entre componentes segue um modelo hierárquico bem definido. Existem dois fluxos principais:



1

Pai → Filho

Usando o decorator `@Input()`, o componente pai passa dados diretamente para o filho via atributos no template.

2

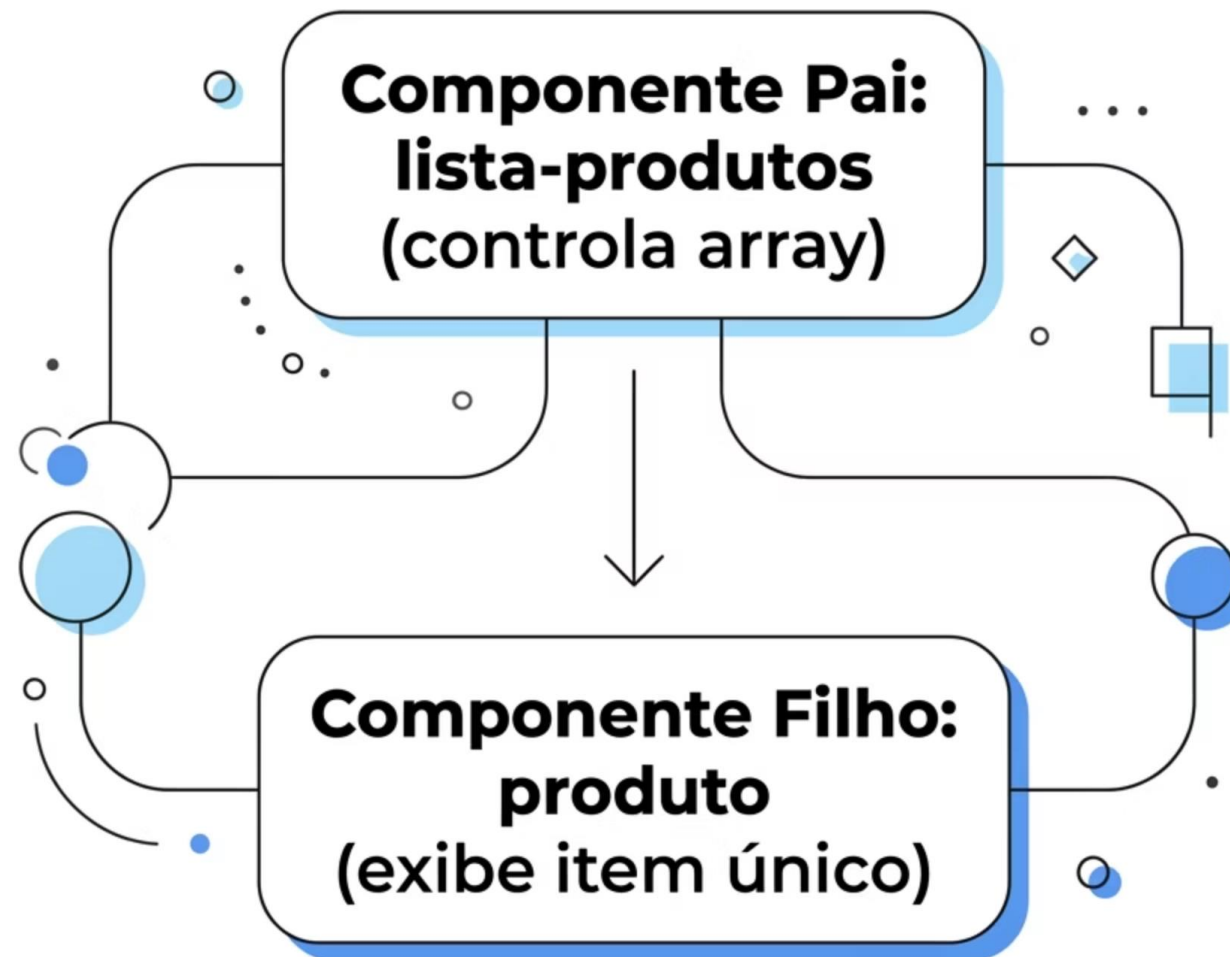
Filho → Pai

Usando `@Output()` com `EventEmitter`, o filho emite eventos que o pai pode escutar e tratar.

Esses dois mecanismos formam a base da comunicação entre componentes em Angular e são fundamentais para qualquer aplicação.

Estrutura do Exemplo Prático

Para ilustrar os conceitos, vamos trabalhar com dois componentes que se relacionam hierarquicamente. Essa estrutura é muito comum em aplicações Angular reais.



O componente `lista-produtos` é o **pai** e gerencia os dados. O componente `produto` é o **filho** e cuida da exibição de cada item individual.

Criando o Componente Pai

O componente `lista-produtos`, que será o componente pai em relação ao componente `produto`, ainda não foi criado.

- 1 - Pare a execução do servidor (`<CTRL> + <C>` no terminal) ;
- 2 – Digite no terminal para criar o componente `lista-produtos`:

```
ng generate component components/lista-produtos
```

Ou, de forma abreviada:

```
ng g c components/lista-produtos
```

 Todos os arquivos do componente `lista-produtos` serão criados na pasta `components`.

- 3 – Após a criação do Componente Pai, rode novamente o servidor:

```
ng serve
```

O Componente Pai Deve Ser Mencionado no App Principal

O componente `lista-produtos`, que será o componente pai deve ser apontado nos arquivos `app.ts` e `app.html`.

Deixe o conteúdo do arquivo `app.ts` assim:

```
import { Component, signal } from '@angular/core';
import { ListaProdutos } from '../components/lista-produtos/lista-produtos';

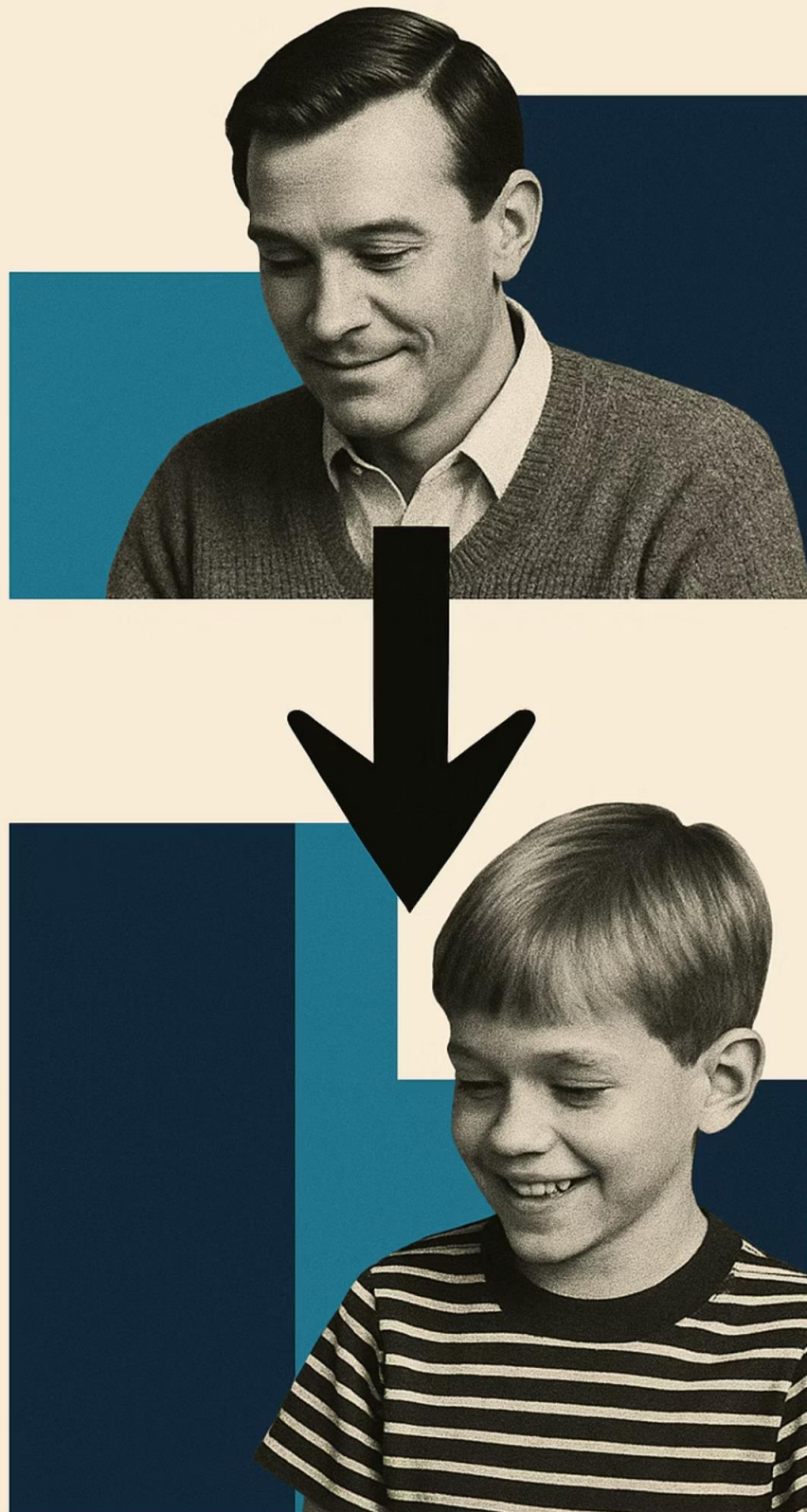
@Component({
  selector: 'app-root',
  imports: [ListaProdutos],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  protected readonly title = signal('meu-primeiro-app');
}
```

O Componente Pai Deve Ser Mencionado no App Principal

Agora é a vez de alterar o template HTML do app principal para mencionar o componente pai.

Deixe o conteúdo do `app.html` assim:

```
<h1>Minha Loja</h1>  
<app-lista-produtos></app-lista-produtos>
```



Comunicação Pai → Filho

@Input()

O decorator `@Input()` é o mecanismo do Angular que permite ao componente **pai enviar dados para o filho**. O filho declara quais propriedades aceita receber, e o pai as preenche via template.

Preparando o Componente Filho

No arquivo `produto.ts`, declare as propriedades que o filho irá receber do pai usando o decorator `@Input()`.

```
import { Component, Input } from '@angular/core';
import { CurrencyPipe } from '@angular/common';

@Component({
  selector: 'app-produto',
  imports: [CurrencyPipe],
  templateUrl: './produto.html',
  styleUrls: ['./produto.css'],
})
export class Produto {
  @Input() nome: string = '';
  @Input() preco: number = 0;
}
```

❗ Cada propriedade com `@Input()` pode receber um valor padrão. Isso evita erros caso o pai não passe o dado.

Preparando o Template do Componente Filho

Deixe o arquivo `produto.html` exatamente assim:

```
<div class="produto">

  <h3>{{ nome }}</h3>
  <p>R$ {{ preco }}</p>

</div>
```

Preparando a Estrutura do Componente Pai

No arquivo de lógica do componente pai (`lista-produtos.ts`), será realizado import do componente filho e inserido dados em uma lista de produtos na classe `ListaProdutos`. Deixe o arquivo `lista-produtos.ts` assim:

```
import { Component } from '@angular/core';
import { Produto } from '../produto/produto';

@Component({
  selector: 'app-lista-produtos',
  imports: [Produto],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})
```

```
export class ListaProdutos {
  produtos = [
    { nome: 'Notebook', preco: 3800 },
    { nome: 'Mouse', preco: 179 }
  ];
}
```

Enviando Dados no Componente Pai

No template do componente pai (`lista-produtos.html`), use a sintaxe de **property binding** com colchetes `[ ]` para passar os dados para cada instância do componente filho.

```
@for (item of produtos; track item.nome) {  
  <app-produto  
    [nome]="item.nome"  
    [preco]="item.preco"  
  ></app-produto>  
}
```

- ❏ A sintaxe `[nome]="item.nome"` é o **property binding**. Os colchetes indicam que o valor é uma expressão TypeScript, não uma string estática.

Exibindo no Componente Filho

No template do componente filho (`produto.html`), use as propriedades recebidas via `@Input()` diretamente na interpolação. O pipe `currency` formata o valor monetário automaticamente.

```
<div class="produto-card">
  <h3>{{ nome }}</h3>
  <p>{{ preco | currency:'BRL' }}</p>
</div>
```

O pipe `currency: 'BRL'` formata o número como moeda brasileira, exibindo o símbolo **R\$** e as casas decimais corretamente.

O que Aconteceu Aqui?

Vamos visualizar o fluxo de dados que acabamos de implementar entre os dois componentes:

Componente Pai (lista-produtos)

- Possui o array completo de produtos
- Itera sobre os itens com `@for`
- Envia dados ao filho via property binding

Componente Filho (produto)

- Recebe os dados via `@Input()`
- Não precisa saber de onde vieram os dados
- Apenas exibe o que recebe no template

Por que Usar @Input?

Além de funcionar, usar `@Input()` corretamente traz benefícios importantes para a arquitetura do seu projeto Angular.



Reutilização de Componentes

O componente filho pode ser usado em qualquer parte da aplicação, recebendo dados diferentes a cada vez.



Separação de Responsabilidades

O pai gerencia os dados, o filho gerencia a exibição. Cada um faz apenas o que é de sua responsabilidade.



Código Mais Organizado

Componentes menores e focados são mais fáceis de ler, testar e manter ao longo do tempo.



Comunicação Filho → Pai

@Output()

Agora vamos fazer o **caminho inverso**. O componente filho precisa notificar o pai sobre algo que aconteceu — como um clique do usuário. Para isso, usamos `@Output()` com `EventEmitter`.

Exemplo Prático

Vamos simular um cenário muito comum em aplicações de e-commerce: o usuário clica em um produto e o componente pai é informado sobre qual produto foi selecionado.

1

Usuário clica no produto

O botão "Selecionar" dentro do componente filho é clicado.

2

Filho emite o evento

O `EventEmitter` dispara um evento carregando o nome do produto.

3

Pai recebe e age

O componente pai escuta o evento e executa sua lógica (ex: exibir alerta, atualizar estado).

Criando o Evento no Filho

No arquivo `produto.ts`, declare um `EventEmitter` com o decorator `@Output()` e crie um método que emite o evento quando chamado.

```
import { Component, Input, Output, EventEmitter } from '@angular/core';
import { CurrencyPipe } from '@angular/common';

...

export class Produto {
  @Input() nome: string = '';
  @Input() preco: number = 0;
  @Output() produtoSelecionado = new EventEmitter<string>();

  selecionarProduto() {
    this.produtoSelecionado.emit(this.nome);
  }
}
```

 O tipo genérico `EventEmitter<string>` indica que o evento vai transportar um valor do tipo `string`.

Disparando o Evento no Template

No template do filho (`produto.html`), conecte o evento de clique do botão ao método `selecionarProduto()` usando a sintaxe de **event binding** com parênteses.

```
<div class="produto-card">
  <h3>{{ nome }}</h3>
  <p>{{ preco | currency:'BRL' }}</p>

  <button (click)="selecionarProduto()">
    Selecionar {{ nome }}
  </button>
</div>
```

(click) é um evento nativo do HTML (DOM), enquanto (produtoSelecionado) é um evento personalizado criado no Angular com @Output.

- ❑ A sintaxe `(click)="metodo()"` é o **event binding** do Angular. Os parênteses indicam que estamos ouvindo um evento do DOM.

Capturando o Evento no Pai

No template do pai (`lista-produtos.html`), use a sintaxe de event binding com parênteses para ouvir o evento customizado emitido pelo filho. Use `$event` para capturar o valor transportado.

```
@for (item of produtos; track item.nome) {  
  <app-produto  
    [nome]="item.nome"  
    [preco]="item.preco"  
    (produtoSelecionado)="exibirProduto($event)"  
  ></app-produto>  
}
```

Note que `(produtoSelecionado)` corresponde exatamente ao nome da propriedade `@Output()` declarada no componente filho.

OBS: `exibirProduto()` ficará com um alerta de erro, que será resolvido com a implementação da próxima ação.

Tratando o Evento no Pai

No arquivo TypeScript do pai (`lista-produtos.ts`), implemente o método que será chamado quando o evento for recebido. O parâmetro `nome` carrega o valor emitido pelo filho.

```
export class ListaProdutos {  
  produtos = [  
    { nome: 'Notebook', preco: 3800 },  
    { nome: 'Mouse', preco: 179 },  
  ];  
  
  exibirProduto(nome: string) {  
    console.log('Produto selecionado:', nome);  
    // Aqui você pode atualizar o estado, abrir modal, etc.  
  }  
}
```

✔ Abra o DevTools do navegador (F12) na aba Console para ver a mensagem sendo exibida ao clicar em um produto.

O que Aconteceu Agora?

Veja o fluxo completo da comunicação filho → pai que acabamos de implementar:

Componente Filho (produto)

- O usuário clica no botão "Selecionar"
- O método `selecionarProduto()` é chamado
- O `EventEmitter` emite o nome do produto

Componente Pai (lista-produtos)

- Está ouvindo o evento `(produtoSelecionado)`
- Recebe o valor via `$event`
- Executa o método `exibirProduto()`

Faça um teste

Com a aplicação em execução no servidor (ng serve):

No navegador

- Visualize a aplicação em `localhost:4200`
- Abra o DevTools pressionando `<F12>`
- Selecione a aba `Console`
- Clique nos botões de Selecionar Produto e veja o resultado no Console.

EventEmitter em Detalhe

O `EventEmitter` é a classe do Angular responsável por criar canais de comunicação do filho para o pai. Entenda suas responsabilidades:

Emitir Eventos

Através do método `.emit(valor)`, o `EventEmitter` dispara um evento que pode ser capturado pelo componente pai no template.

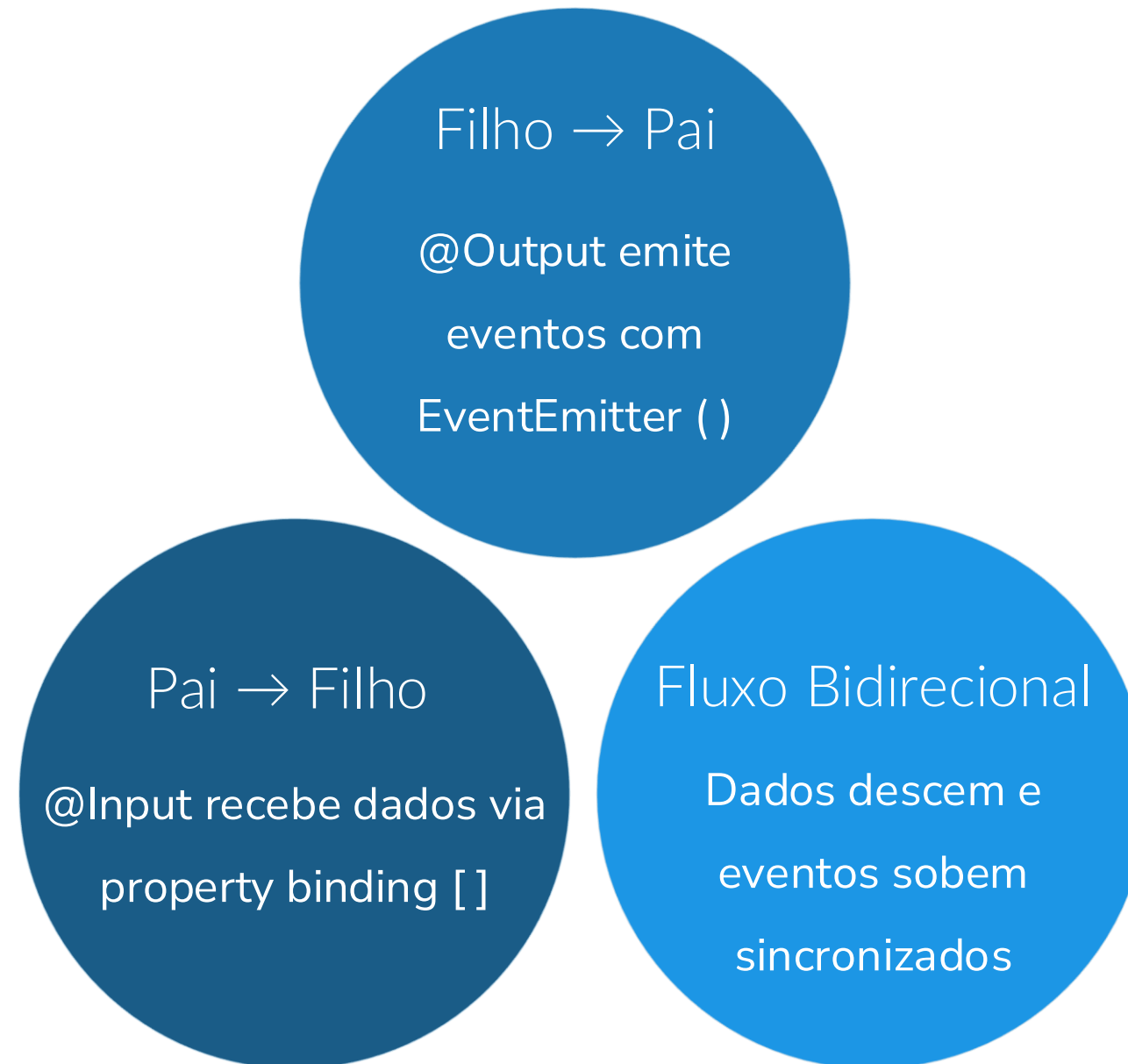
Transportar Dados

O evento pode carregar qualquer valor junto — uma string, um número, um objeto. O tipo é definido no genérico: `EventEmitter<Tipo>`.

- ❗ Sempre use o tipo genérico ao declarar o `EventEmitter`. Isso garante que o TypeScript valide o tipo do dado transportado.

Fluxo Completo de Comunicação

Agora que você conhece os dois mecanismos, veja o resumo visual do fluxo completo entre componentes pai e filho em Angular:



Boas Práticas

Além de funcionar corretamente, é importante escrever código que seja fácil de entender e manter. Siga estas boas práticas ao trabalhar com `@Input` e `@Output`:

Não acople lógica entre componentes

O componente filho não deve conhecer detalhes internos do pai. Comunique-se apenas por `@Input` e `@Output`, nunca injetando o pai diretamente.

Mantenha responsabilidades separadas

O pai gerencia o estado e os dados. O filho gerencia a apresentação e as interações do usuário. Misturar essas responsabilidades gera código difícil de manter.

Use nomes claros para eventos

Prefira nomes descritivos como `produtoSelecionado` ou `itemRemovido` em vez de nomes genéricos como `clicked` ou `evento1`.

Organização do Projeto

Ao implementar a comunicação entre componentes, seu projeto Angular ganhou uma estrutura bem mais sólida e escalável.



Componente de Lista

`lista-produtos`: controla o estado, possui os dados e coordena os filhos.



Componente de Item

`produto`: responsável por exibir um único item e capturar interações do usuário.



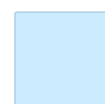
Comunicação Estruturada

Dados fluem do pai para o filho via `@Input` e eventos sobem via `@Output`.



Evolução do Projeto

Compare onde você estava e onde chegou. Seu sistema Angular evoluiu significativamente nesta aula:



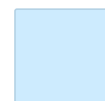
Exibe produtos dinamicamente

Os dados são renderizados em lista usando `@for`, sem duplicação de código HTML.



Permite interação do usuário

O clique em um produto dispara um evento que o pai captura e pode processar.



Simula comportamento real

A estrutura já se parece com o que você encontrará em aplicações Angular profissionais.

Preparação para o Futuro

Com o domínio de `@Input` e `@Output`, você está preparado para entender as abordagens mais modernas que o Angular oferece.

Signals (Angular 16+)

Uma forma mais moderna e reativa de gerenciar e compartilhar estado entre componentes, com melhor performance e detecção de mudanças mais eficiente.

O conceito é o mesmo

Mesmo com Signals ou outras abordagens avançadas como serviços e RxJS, o conceito fundamental de comunicação entre componentes permanece o mesmo. O que você aprendeu hoje é a base de tudo.

- ❏ Não se preocupe com Signals agora. Foque em dominar `@Input` e `@Output` primeiro. Signals virão naturalmente depois.

O que Aprendemos até Aqui

Faça uma pausa e consolide os conhecimentos adquiridos nesta aula. Você agora domina os fundamentos da comunicação entre componentes Angular.

01

Comunicação Pai → Filho com @Input

Enviar dados do componente pai para o filho usando property binding e o decorator `@Input()`.

03

Uso do EventEmitter

Criar um canal de eventos customizados que transportam dados entre componentes.

02

Comunicação Filho → Pai com @Output

Emitir eventos do filho para o pai usando o decorator `@Output()` e event binding.

04

Organização de Componentes

Estruturar o projeto com responsabilidades claras e comunicação bem definida.

O que Não Precisa se Preocupar Agora

É natural se sentir sobrecarregado ao aprender Angular. Mas parte do aprendizado eficiente é saber o que **não** precisa dominar neste momento.

Decorar toda a sintaxe

Você pode consultar a documentação quando precisar. O importante é entender o *conceito* de como a comunicação funciona.

Todos os cenários possíveis

Existem casos avançados como múltiplos níveis de componentes e comunicação por serviços. Esses tópicos virão nas próximas aulas.

Dominar Signals agora

Signals são um recurso moderno do Angular. Você aprenderá no momento certo, com a base que está construindo agora.

- ✔ Foque no que foi ensinado hoje. A prática constante consolida o aprendizado melhor do que tentar absorver tudo de uma vez.



Parabéns!

Você chegou lá!

Agora você sabe fazer componentes Angular conversarem entre si usando `@Input` e `@Output`. Esse é um marco fundamental no desenvolvimento com Angular — **essencial para qualquer aplicação real**.

`@Input`

Pai → Filho

`@Output`

Filho → Pai

EventEmitter

Transporta dados

Dúvidas

?

